Python

• Lab 6–7 • , dictionary, input

- input ⟶ output ⟶
dict, list, int str.

- assert
return; print .

- ⟶ ⟶
⟶ return .

Dictionary: key value

```
# -> ( , )
courses = {'2301117': ('Calculus I', '4')}
info = courses.get(course_id)
if info is None: return 'Unknown'
return info[0] + ' ' + info[1]
```

- : d[key] d.get(key); key: if key in d.
str.

```
counts = {}
with open(file_name) as f:
    for line in f:
        for word in line.split():
            word = word.strip(':?,.')
            counts[word] = counts.get(word, 0) + 1
return counts
```

- split() ; strip(chars)

.

:

```
scores = {}
with open(file_name) as f:
    for line in f:
        name, score = line.split()
        scores[name] = int(score)
return scores

# ->
return sorted(scores, key=scores.get, reverse=True)
```

- / int. sorted(..., key=..., reverse=True)
.

Top 3 / /

```
top3 = sorted(scores, key=scores.get, reverse=True)[:3]
popular = [name for name, n in sports.items() if n >= 15]
total = sum(sports.values())
```

- : total = int(a) + int(b) + int(c) + int(audience).

Dictionary : stock / menu /

```
# stock[id][size] = quantity
if product_id not in stock:
    stock[product_id] = {}
stock[product_id][size] = stock[product_id].get(size, 0) + qty

# menu[drink][size] = price
total = menu[drink][size] * number
```

- : key ? value
dict ? id int size str.

input

```
drink, size, number = input('...').split()
number = int(number)
if drink not in menu:
    print('Drink not available.')
elif size not in menu[drink]:
    print('Incorrect size.')
else:
    print(f'Total : {menu[drink][size] * number}')
```

-
.
.

```
faculty = student_id[-2:]  # 2

level_digit = student_id[2]  #  3
year = int(student_id[:2])

# map  group / level / term / id_group
fee = table[group][level][term][id_group]
```

- string slice. dict
lookup .

- , int , handle
/ , .

- assert : , colon,
newline .

- key dict values; key
.

```
 -> split/  -> dict
dict -> loop/  -> list
input ->  -> lookup -> print
```

: , , ,
, stock, , ,

OOP:

• Lab 8 • , attribute, method object collection

- noun → class. → attribute. → method.

- Class = ; object = . self object method.

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width

r = Rectangle(5, 3)
r.area()  # self r
```

__init__

- Rectangle(5, 3) object __init__(object, 5, 3) ; self.attribute = parameter.

- obj.method(x) Class.method(obj, x); self .

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author  # object Author

# author.name Author
# book.title Book
```

- : book → book.author → book.author.name. object .attribute [0].

/

```
# cart[item]
self.cart[item] = self.cart.get(item, 0) + qty
self.cart[item] -= qty
if self.cart[item] <= 0:
    del self.cart[item]
return sum(self.cart.values())
```

Bank ✦ Account

```
# Bank.accounts: name -> Account object
self.accounts[name] = Account(balance)
self.accounts[src].withdraw(amount)
self.accounts[dst].deposit(amount)

for name, account in self.accounts.items():
    if account.get_balance() < 0:
        debtors.append(name)
```

- accounts[name] object Account; get_balance() __balance.

```
# rate = 1.5 1.5%
interest = balance * rate / 100
if interest >= 0: account.deposit(interest)
else: account.withdraw(-interest)
```

Team: list Member objects

```
for member in self.members:
    if member.name == name:
        self.members.remove(member)
        break

counts = {}
for member in self.members:
    counts[member.role] = counts.get(member.role, 0) + 1
total = sum(counts.values())
pct = {role: round(n / total * 100, 2)
      for role, n in counts.items()}
```

- members object Member: member.name / member.role. remove(name) name str.

Library: dict Book objects

```
# ISBN -> Book object
if book.isbn in self.books:
    self.books[book.isbn].available_copies += book.available_copies
else:
    self.books[book.isbn] = book

total = sum(b.available_copies for b in self.books.values())
found = [b for b in self.books.values()
        if b.author.name == author_name]
```

__str__

```
def __str__(self):
    return f'Name: {self.name}, Role: {self.role}'

# print(obj) __str__
# display method: return print
# \n
```

- assert string newline / . fail.

Checklist OOP

- attribute __init__; self object.

- object (has-a). object reference .

- returns return. 0 / .

:

__init__ → method → → return

test